DISEÑO Y FABRICACIÓN DE UN SISTEMA DE TELEMETRÍA EN TIEMPO REAL CON CONECTIVIDAD MÓVIL

Autor:
JULIÁN SAMBLÁS CABALLERO
Tutor:
JAVIER PÉREZ FERNÁNDEZ

GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL
Trabajo Fin de Grado
Junio 2024

ÍNDICE

1 Introducción

2 Telemetría y adquisición de datos

3 Componentes del dispositivo

4 Prototipos

5 Métodos de adquisición y visualización de datos

6 Montaje y pruebas realizadas al dispositivo en funcionamiento

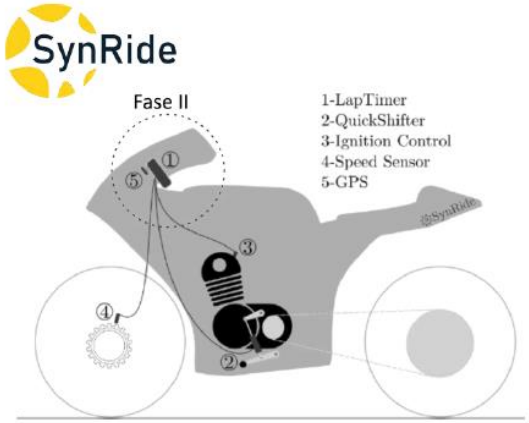
7 Conclusiones y futuro del proyecto

1. INTRODUCCIÓN

Antecedentes:

Programa Synride

- *Pertenece al programa de maduración de proyectos UMA*
- *Tecnología del automovilismo de competición al usuario amateur*



- *Tiempo por vuelta*
- *QuickShifter*
- *Control de encendido*
- *Sensor de velocidad*
- *GPS*

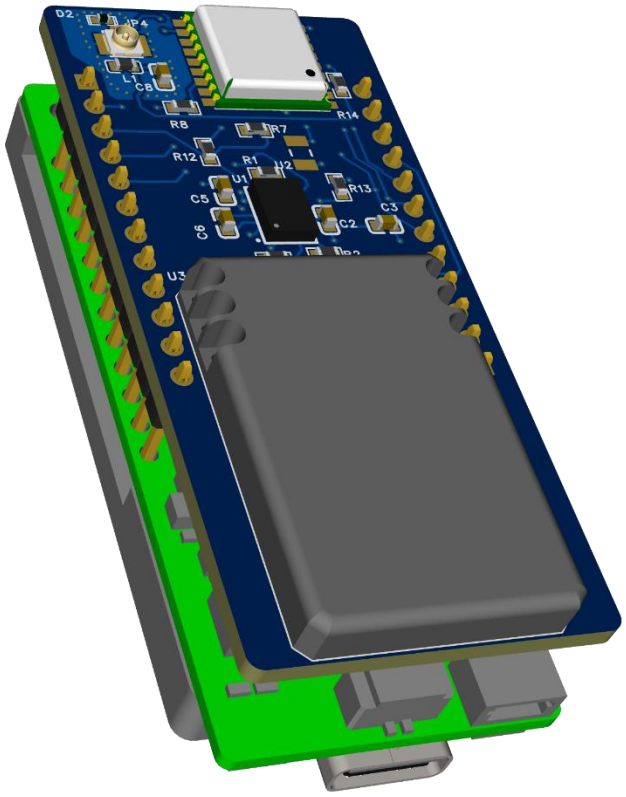
Modelo anterior

1. INTRODUCCIÓN

Objetivos:

Construir y programar un dispositivo capaz de medir:

- *Localización (GPS)*
- *Aceleraciones, ángulos de giro y velocidad(IMU)*
- *Transmisión de datos (SIM)*



4 Módulos diferenciados

PANTALLA

GPS

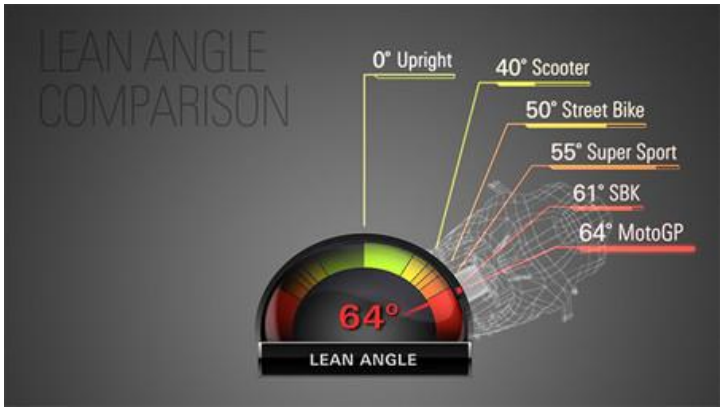
TARJETA SIM

IMU

Adquisición de datos → *Visualización en tiempo real*

2. TELEMETRÍA Y ADQUISICIÓN DE DATOS

1) *¿Qué es la telemetría y adquisición de datos?*
2) *Estado del arte*
3) *Importancia de la adquisición de datos en la actualidad*
4) *Importancia de la adquisición de datos en el mundo del motor*
5) *Telemetría aplicada a la competición*
6) *Objetivos del sistema proyectado*

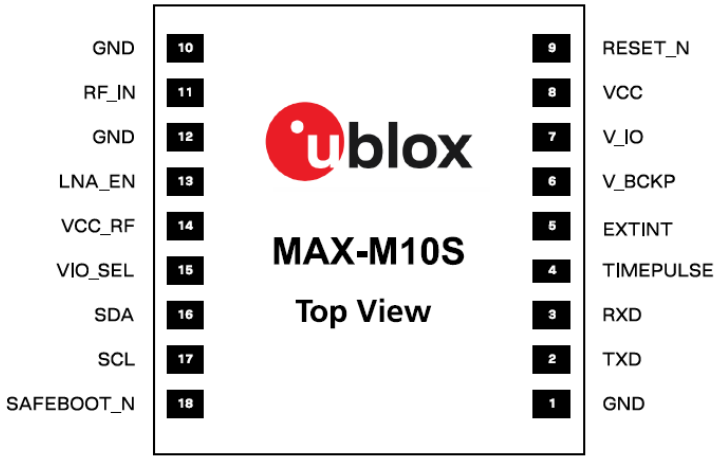




motorec

MÓDULO GPS

- Sistema de posicionamiento global
- Funcionalidades del módulo GPS
- Opciones de mercado



Opciones de mercado

Modelo	Precio unitario (€)	Precio x10 (€)	Consumo en funcionamiento modo tracking (continuous mode)(mW)	IMU	Frecuencia de rastreo (Hz)
MAX-M10S	19,53	16,24	25	No	25
NEO-M9N-00B	25,11	22,25	108 (Variable*)	No	25
EVA-M8M	26,97	13,67	66	No	18
SAM-M10Q	29,30	20,88	30	No	25
MIA-M10Q	26,97	13,67	25	No	25
Teseo-VIC3DA	20,50	18,95	193	Si	30

UNIDAD DE MEDIDA INERCIAL (IMU)

- Desempeña un papel crucial
- Medición de aceleración y velocidad angular
- Análisis de inclinación
- Sistemas de estabilización electrónica



Opciones de mercado

Modelo	Precio unitario (€)	Precio x10 (€)	Ejes axiales
BMI323	4,46	2,86	6
BHI360	6,08	4,87	6
ASM330LHHXTR	13,02	10,30	6
LSM6DSV16XTR	6,29	5,03	6
BNO055	11,56	8,65	9

MÓDULO SIM

- Enviar la información obtenida a través de internet
- Ocupación de espacio mínimo en la placa PBC
- Capacidad de operar en una banda 4G

Opciones de mercado

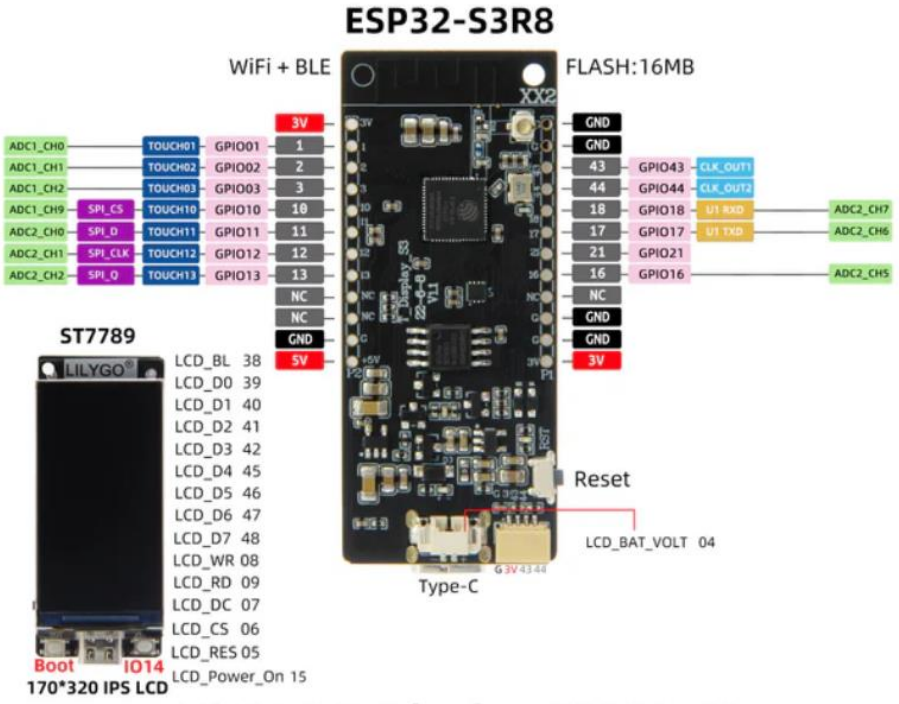


DISPLAY

- Calidad de imagen nítida
- Microprocesador ESP32-S3
- Fácil integración en diversos proyectos gracias a sus pines
- Versátil programación



Lilygo T-Display S3



4. PROTOTIPOS

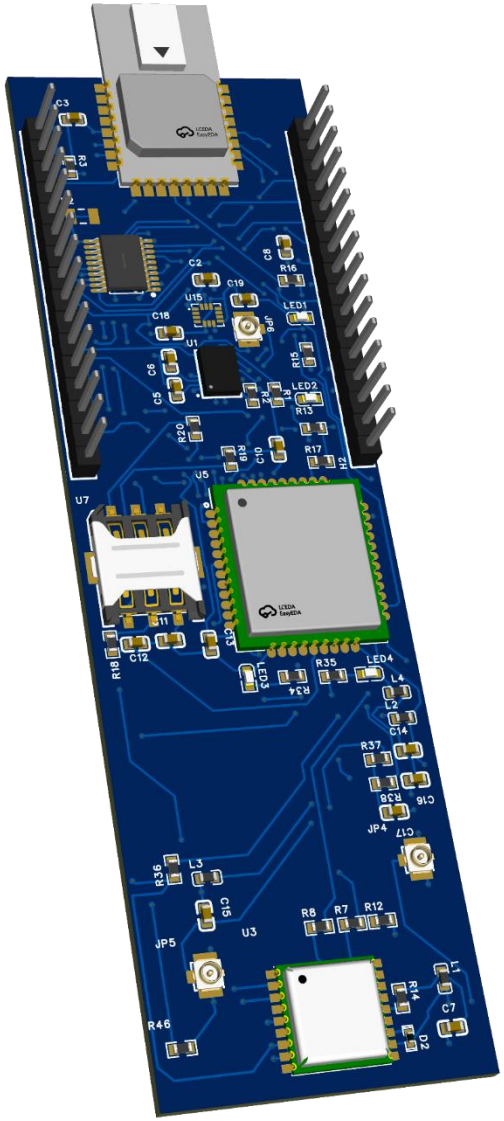
PRIMER PROTOTIPO

Basado en la placa T-SIM7080G-S3

- *Potente procesador*
- *Incorporación del módulo SIM*

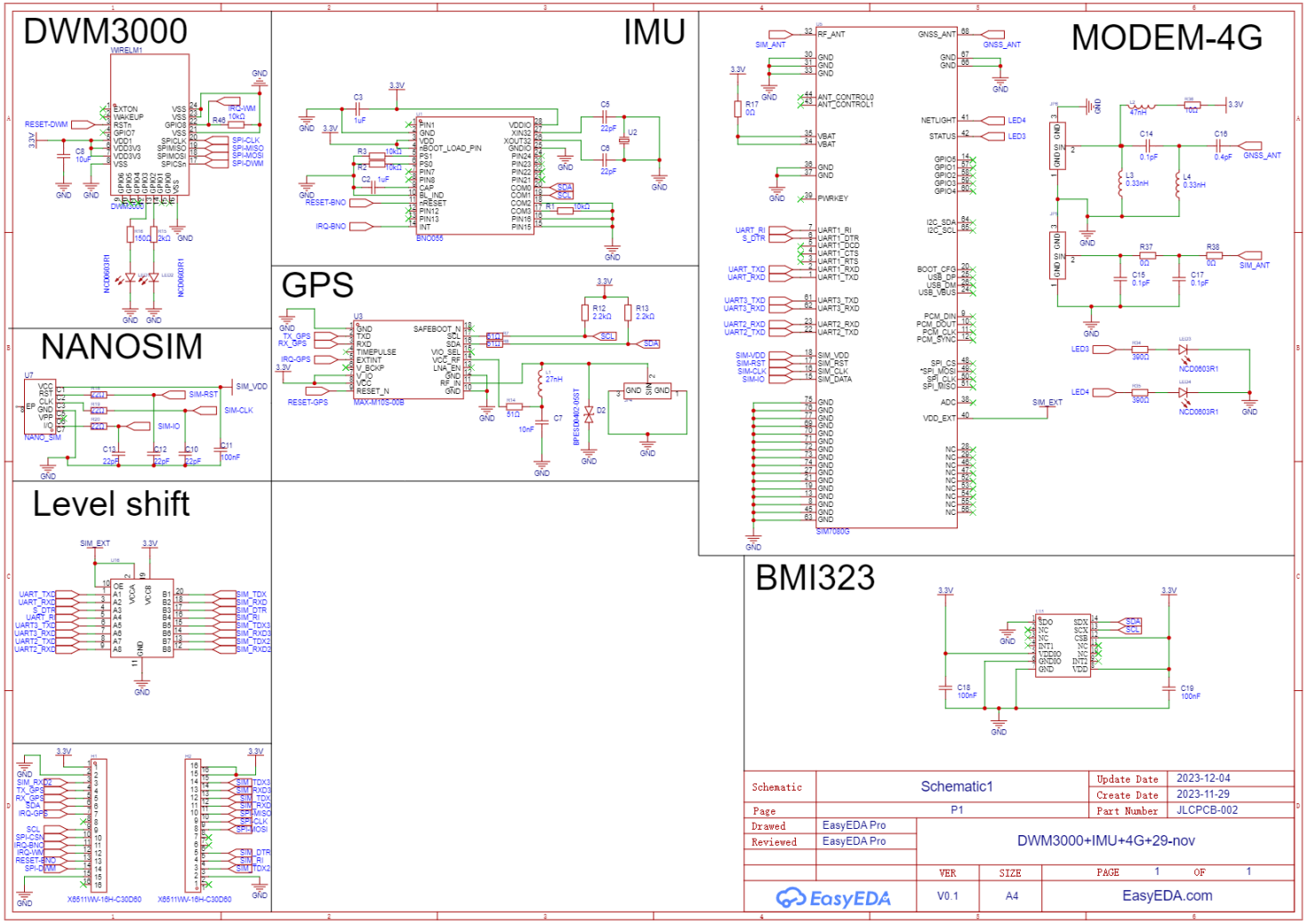
Módulos acoplados

- *Dos sistemas de posicionamiento: MAX-M10S y DWM3000*
- *Dos sistemas inerciales de medida: BNO055 y BMI323.*



4. PROTOTIPOS

PRIMER PROTOTIPO



4. PROTOTIPOS

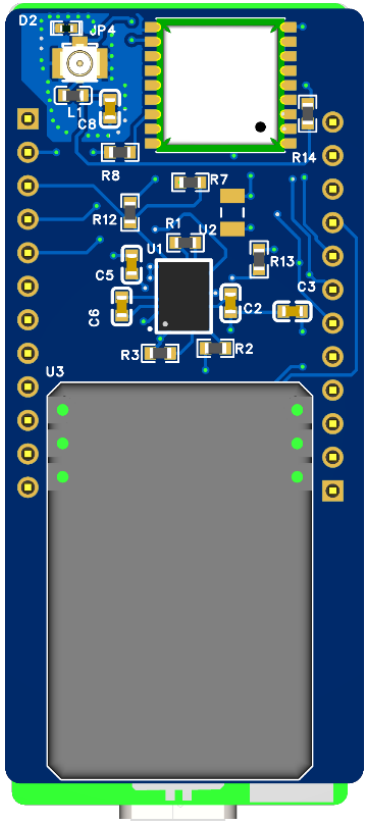
SEGUNDO PROTOTIPO

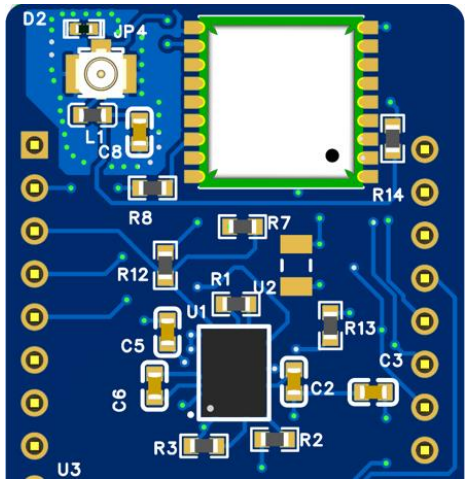
Basado en el dispositivo T-DISPLAY-S3

- *Potente procesador*
- *Menor tamaño*
- *Acople de display*

Módulos acoplados

- *Sistema de posicionamiento: MAX-M10S*
- *Sistema inercial de medida: BNO055*
- *Módulo SIM: SIM7080*

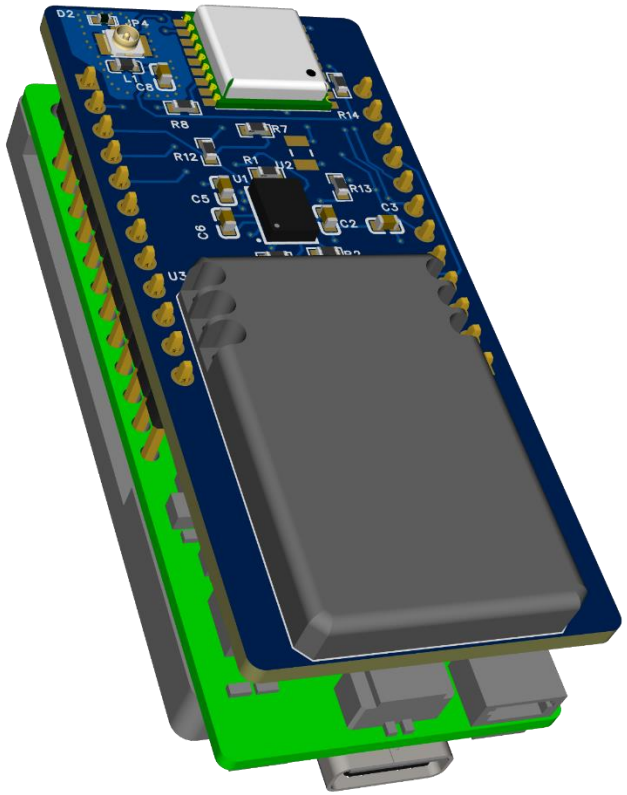


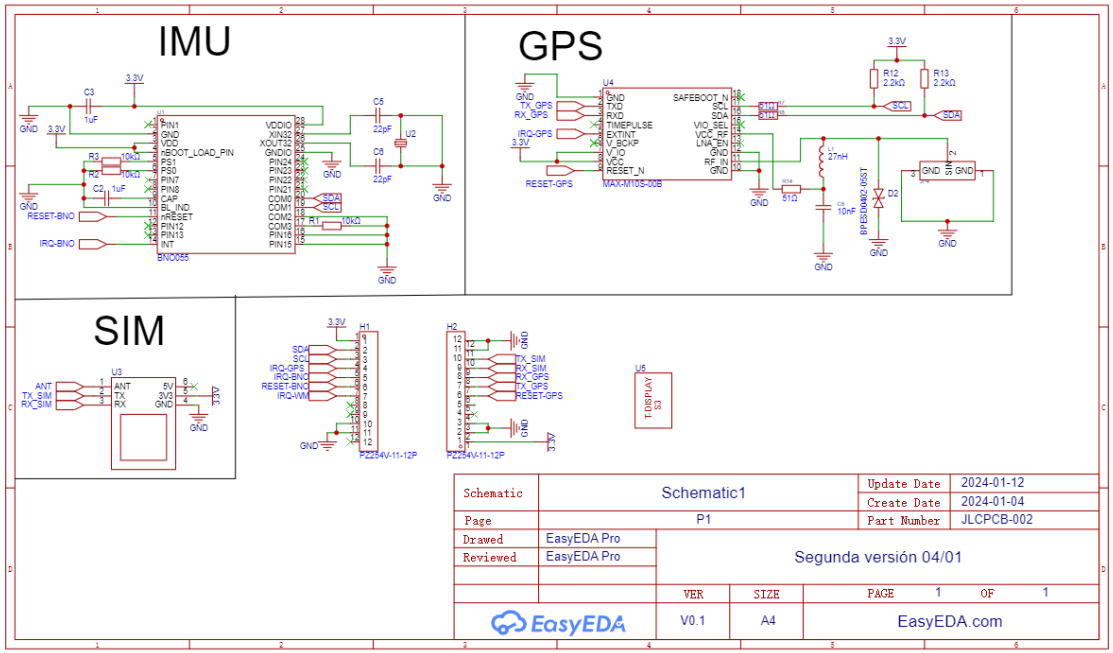


4. PROTOTIPOS

SEGUNDO PROTOTIPO

Basado en el dispositivo T-DISPLAY-S3

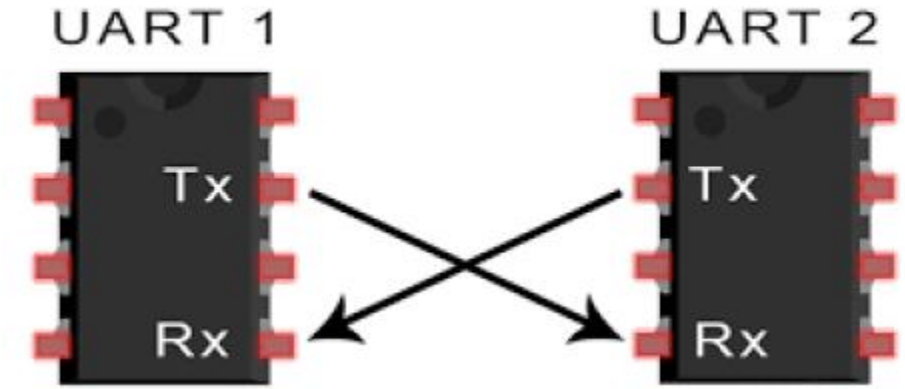




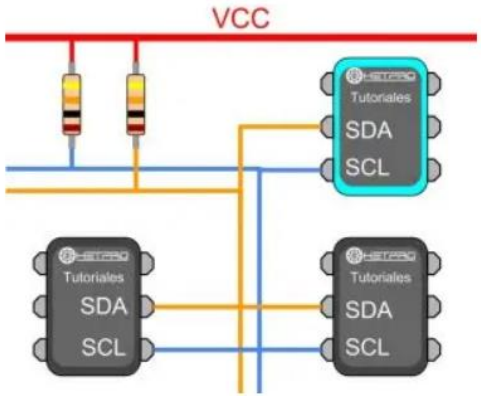
PROTOCOLOS DE COMUNICACIÓN UTILIZADOS

GPS →

SIM →



IMU →



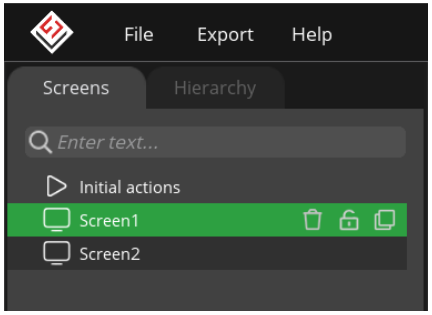
PROGRAMACIÓN EN SQUARE LINE STUDIO Y ARDUINO IDE

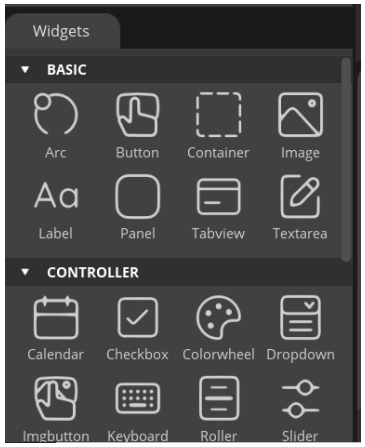
SQUARE LINE STUDIO

- Programación más intuitiva y rápida que de manera convencional
- Facilidad de adaptación a Arduino IDE gracias a la librería LVGL



Configurador de diseños

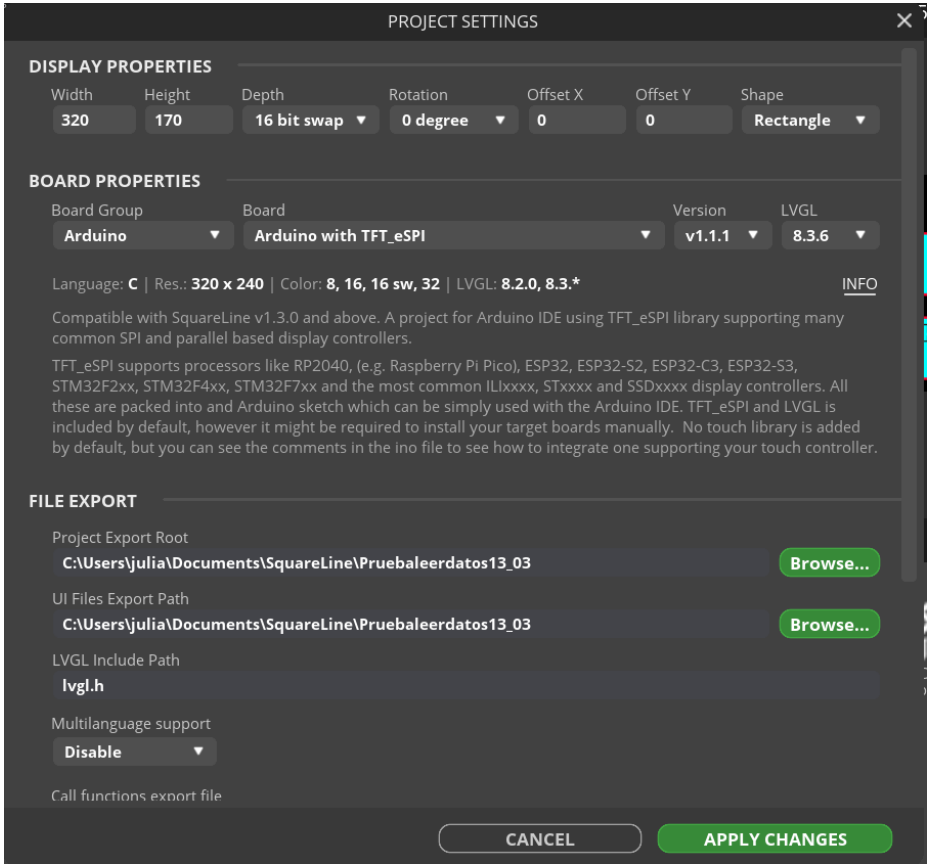




PROGRAMACIÓN EN SQUARE LINE STUDIO Y ARDUINO IDE

SQUARE LINE STUDIO

- Configuración de exportación de archivos
- Profundidad de colores de la pantalla
- Ancho y largo de pantalla
- Versión de la librería



PROGRAMACIÓN EN SQUARE LINE STUDIO Y ARDUINO IDE

Arduino IDE

- Adaptación de los archivos exportados desde Square Line Studio

- Añadir el archivo pin.h extraído de Lilygo para la configuración de los pines del Display

- Estructura del código principal: inclusión de librerías, declaración de variables, funciones auxiliares, función *setup(),* función *loop().*



ARDUINO IDE

Inclusión de librerías

```
#define LCD_MODULE_CMD_1
#define USE_UI

#include "OneButton.h"
#include "lvgl.h"
#include <TFT_eSPI.h>
#include "Arduino.h"

#include "Wire.h"
#include "esp_lcd_panel_io.h"
#include "esp_lcd_panel_ops.h"
#include "esp_lcd_panel_vendor.h"
#include "ui.h"
#include "pin_config.h"
#include "esp_sntp.h"
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>
#include <utility/imumaths.h>
#include <SparkFun_u-blox_GNSS_v3.h>
SFE_UBLOX_GNSS myGNSS;

```

Declaración de variables

```
const int GNSS_RX_PIN = 17;
const int GNSS_TX_PIN = 18;

long lastTime = 0;

int Screen=1;
double linearAccelDataX=0;
double linearAccelDataY=0;
double linearAccelDataZ=0;
double latitude;
int altitude;
//int speedAccEst;
double longitude;
double SIV;
int nedNorthVel;
int nedEastVel;
int nedDownVel;
```

ARDUINO IDE

Configuración del display

```
bool enabled=0;
esp_lcd_panel_io_handle_t io_handle = NULL;
static lv_disp_draw_buf_t disp_buf;
static lv_disp_drv_t disp_drv;
static lv_color_t *lv_disp_buf;
static bool is_initialized_lvgl = false;
OneButton button1(PIN_BUTTON_1, true);
OneButton button2(PIN_BUTTON_2, true);
#if defined(LCD_MODULE_CMD_1)
typedef struct {
    uint8_t cmd;
    uint8_t data[14];
    uint8_t len;
} lcd_cmd_t;

lcd_cmd_t lcd_st7789v[] = {
    {0x11, {0}, 0 | 0x80},
    {0x3A, {0X05}, 1},
    {0xB2, {0X0B, 0X0B, 0X00, 0X33, 0X33}, 5},
    {0xB7, {0X75}, 1},
    {0xBB, {0X28}, 1},
    {0xC0, {0X2C}, 1},
    {0xC2, {0X01}, 1},
    {0xC3, {0X1F}, 1},
    {0xC6, {0X13}, 1},
```

Función de cambio de pantalla

```
void IRAM_ATTR toggleLCD()
{
  if (Screen==1){
   lv_disp_load_scr(ui_Screen2);
   Screen=2;
  }else{
  lv_disp_load_scr(ui_Screen1);
  Screen=1;
  }
}
```

ARDUINO IDE

Función setup()

```
pinMode(PIN_LCD_RD, OUTPUT);
digitalWrite(PIN_LCD_RD, HIGH);
esp_lcd_i80_bus_handle_t i80_bus = NULL;
esp_lcd_i80_bus_config_t bus_config = {
    .dc_gpio_num = PIN_LCD_DC,
    .wr_gpio_num = PIN_LCD_WR,
    .clk_src = LCD_CLK_SRC_PLL160M,
    .data_gpio_nums =
    {
        PIN_LCD_D0,
        PIN_LCD_D1,
        PIN_LCD_D2,
        PIN_LCD_D3,
        PIN_LCD_D4,
        PIN_LCD_D5,
        PIN_LCD_D6,
        PIN_LCD_D7,
    },
    .bus_width = 8,
    .max_transfer_bytes = LVGL_LCD_BUF_SIZE * sizeof(uint16_t),
    .psram_trans_align = 0,
    .sram_trans_align = 0
};
```

Función setup()

```
    ui_init();
    ui_reset();

attachInterrupt(PIN_BUTTON_2, toggleLCD, RISING);
  //GPSINIT
  GPSinit();
  //BNOINIT
  BNOinit();
```

ARDUINO IDE

```
void GPSinit()
{

  Wire.begin(1,2);

  if (myGNSS.begin() == false) //Connect to the u-blox module using Wire port
  {
    Serial.println(F("u-blox GNSS not detected at default I2C address. Please check wiring. Freezing."));
    while (1);
  }

  myGNSS.setI2COutput(COM_TYPE_UBX); //Set the I2C port to output UBX only (turn off NMEA noise)
  myGNSS.setNavigationFrequency(2); //Produce two solutions per second
  myGNSS.setAutoPVT(true); //Tell the GNSS to "send" each solution
  myGNSS.saveConfiguration(); //Optional: Save the current settings to flash and BBR
}
```

Función GPS()

```
void BNOinit(){

  Serial.println("Orientation Sensor Test"); Serial.println("");
  Wire.begin(1,2);
  /* Initialise the sensor */
  if (!bno.begin())
  {
    /* There was a problem detecting the BNO055 ... check your connections */
    Serial.print("Ooops, no BNO055 detected ... Check your wiring or I2C ADDR!");
    while (1);
  }

  delay(1000);
}
```

Función BNO()

ARDUINO IDE

Función loop()

```
void loop()
{
  lv_timer_handler();

  delay(10);

  if (millis()>2000 && enabled==0){

   enabled=1;
  }
 //GPSINIT
    //Query module only every second. Doing i
 //The module only responds when a new positio
 if (millis() - lastTime > 1000)
 {
  lastTime = millis(); //Update the timer

  latitude = myGNSS.getLatitude();
  Serial.print(F("Lat: "));
  Serial.print(latitude);

  longitude = myGNSS.getLongitude();
  Serial.print(F(" Long: "));
  Serial.print(longitude);
  Serial.print(F(" (degrees * 10^-7)"));
```

Función loop()

```
void printEvent(sensors_event_t* event) {
  double x = -1000000, y = -1000000 , z = -1000000;
  if (event->type == SENSOR_TYPE_ACCELEROMETER) {
    Serial.print("Accl:");
    x = event->acceleration.x;
    linearAccelDataX=event->acceleration.x;
    y = event->acceleration.y;
    linearAccelDataY=event->acceleration.y;
    z = event->acceleration.z;
    linearAccelDataZ=event->acceleration.z;
  }
```

```
//BNOINIT
  //could add VECTOR_ACCELEROMETER, VECTOR_MAGNETOMETER,VECTOR_GRAVITY...
sensors_event_t orientationData , angVelocityData , linearAccelData, magne
bno.getEvent(&orientationData, Adafruit_BNO055::VECTOR_EULER);
bno.getEvent(&angVelocityData, Adafruit_BNO055::VECTOR_GYROSCOPE);
bno.getEvent(&linearAccelData, Adafruit_BNO055::VECTOR_LINEARACCEL);
bno.getEvent(&magnetometerData, Adafruit_BNO055::VECTOR_MAGNETOMETER);
bno.getEvent(&accelerometerData, Adafruit_BNO055::VECTOR_ACCELEROMETER);
bno.getEvent(&gravityData, Adafruit_BNO055::VECTOR_GRAVITY);

printEvent(&orientationData);
printEvent(&angVelocityData);
printEvent(&linearAccelData);
printEvent(&magnetometerData);
printEvent(&accelerometerData);
printEvent(&gravityData);

int8_t boardTemp = bno.getTemp();
Serial.println();
Serial.print(F("temperature: "));
Serial.println(boardTemp);
```

ARDUINO IDE

```
void updateBNO() {

  lv_label_set_text(ui_Aceleracionxd, String(linearAccelDataX).c_str());

  lv_label_set_text(ui_aceleracionyd, String(linearAccelDataY).c_str());

  lv_label_set_text(ui_Aceleracionzd, String(linearAccelDataZ).c_str());

}
```

Función updateBNO()

```
void updateGPS(){
  int velocidad = sqrt(sq(nedNorthVel) + sq(nedEastVel) + sq(nedDownVel));
  int velocidadkmh = velocidad*0.0036;
  lv_label_set_text(ui_Labelfront, String(velocidadkmh).c_str());
  lv_slider_set_value(ui_Slider1, int(velocidadkmh), LV_ANIM_ON);

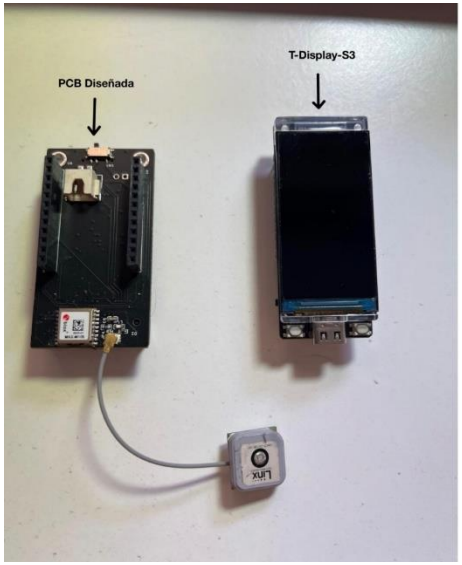
  lv_label_set_text(ui_Satelitesnumero, String(SIV).c_str());

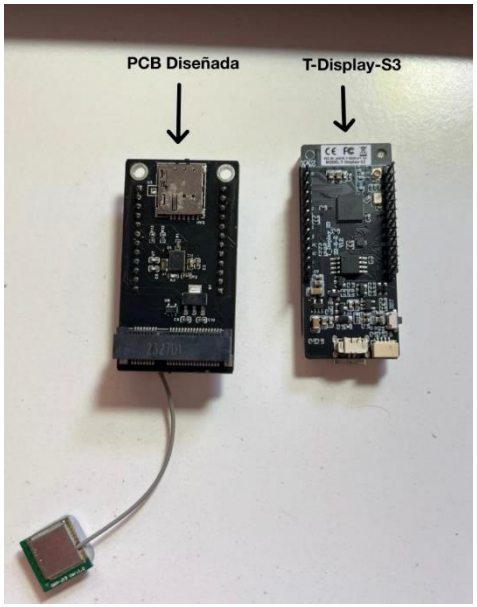
  lv_label_set_text(ui_Gradoslatitudn, String(latitude).c_str());

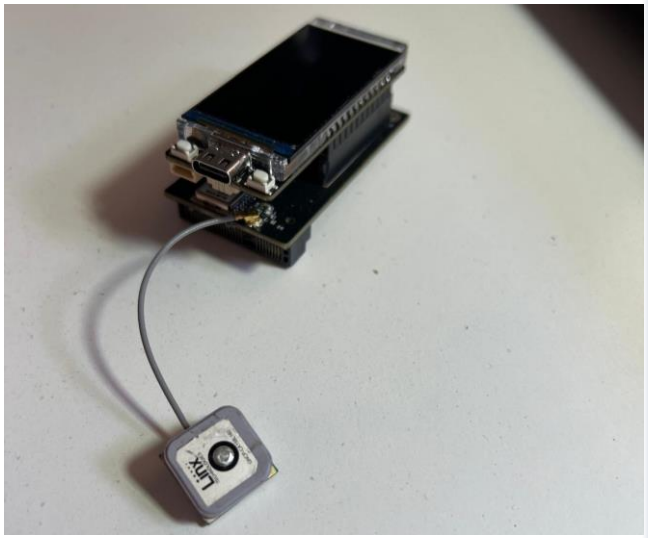
  lv_label_set_text(ui_Gradoslongitudn, String(longitude).c_str());
  int Altitude1=altitude/100;
  lv_label_set_text(ui_Altitudt, String(Altitude1).c_str());
}
```

Función updateGPS()

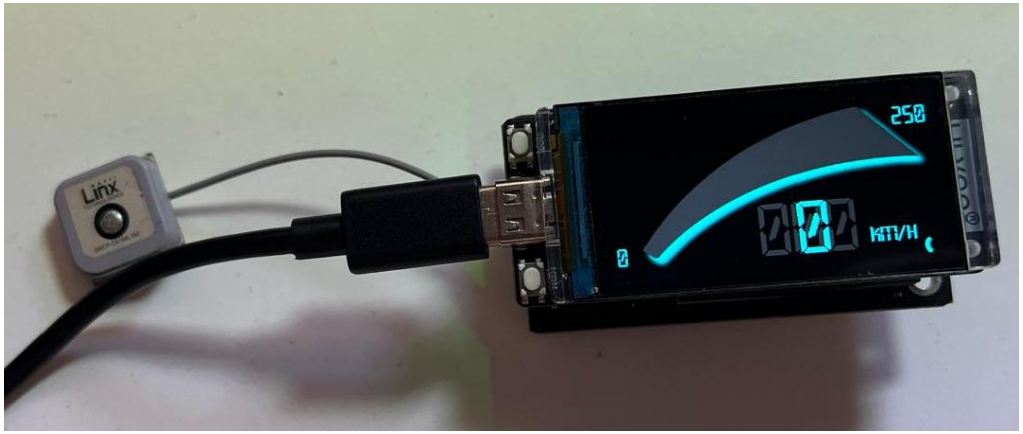
MONTAJE DEL DISPOSITIVO

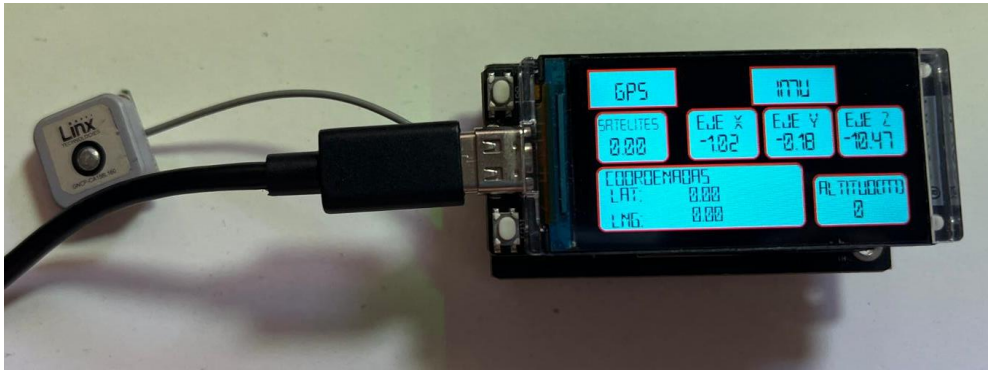






MONTAJE DEL DISPOSITIVO





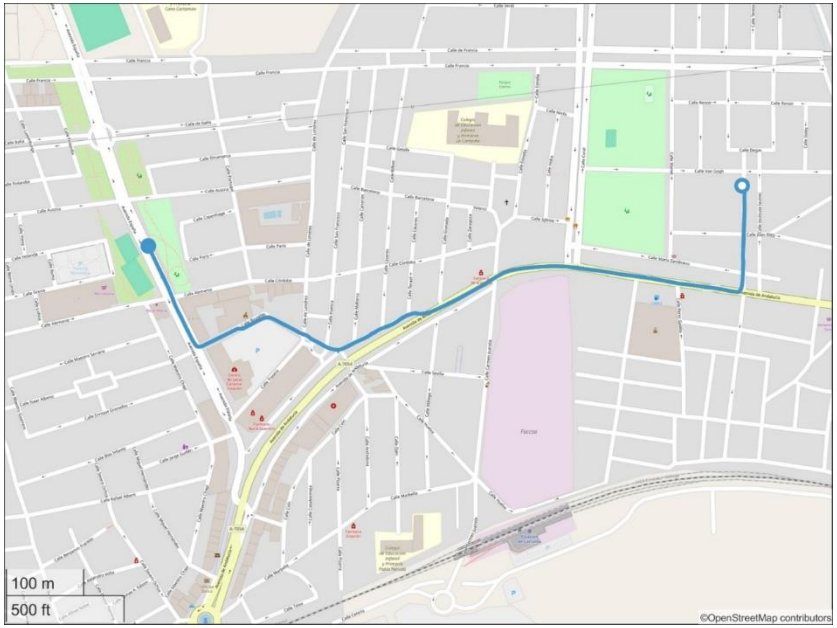
PRIMERA PRUEBA REALIZADA EN FUNCIONAMIENTO





PRUEBA DE TRAYECTORIA

HORA	LAT	LONG	SIV
22:27:10.641	367378578.00	-46053442.00	11
22:27:10.900	367378635.00	-46053429.00	10
22:27:10.900	367378651.00	-46053423.00	11
22:27:10.900	367378665.00	-46053403.00	10
22:27:10.900	367378711.00	-46053378.00	10
22:27:11.922	367378639.00	-46053295.00	10
22:27:12.984	367378457.00	-46053191.00	11
22:27:13.996	367378233.00	-46053125.00	11



Conclusión

- Muestra de datos obtenidos en tiempo real con gran exactitud
- Mejora del dispositivo gracias al segundo prototipo del proyecto
- Impedimentos por falta de información acerca de la pantalla T-Display-S3
- Square Line Studio y LVGL
- Futura visualización de los valores obtenidos

- *Futuro del proyecto:*
- *Enviar los datos obtenidos en tiempo real a una página web llamada SpeedSense*
- *Visualizar estos valores en su contexto*
- *Mejorar las habilidades de conducción del piloto*

DISEÑO Y FABRICACIÓN DE UN SISTEMA DE TELEMETRÍA EN TIEMPO REAL CON CONECTIVIDAD MÓVIL

DISEÑO Y FABRICACIÓN DE UN SISTEMA DE TELEMETRÍA EN TIEMPO REAL CON CONECTIVIDAD MÓVIL

Autor:
JULIÁN SAMBLÁS CABALLERO
Tutor:
JAVIER PÉREZ FERNÁNDEZ

GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL
Trabajo Fin de Grado
Junio 2024